

Session 2.3

Forward Propagation, Backpropagation, and Training

How the parameters of a network are chosen, one step computed by hand, and when to stop



Zhigang Feng

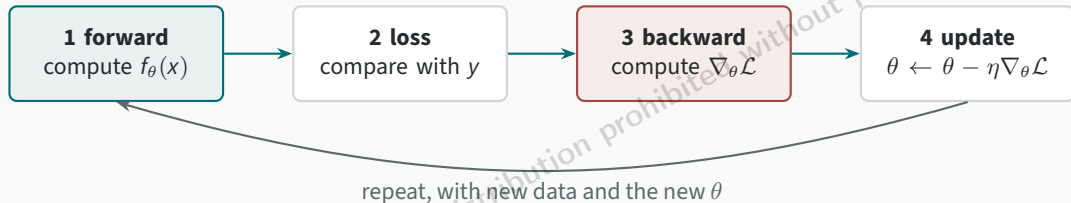
The Problem, Stated

- ▶ Deck 2.2 fixed a family of functions f_θ and showed that a good member exists. Training chooses θ :

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

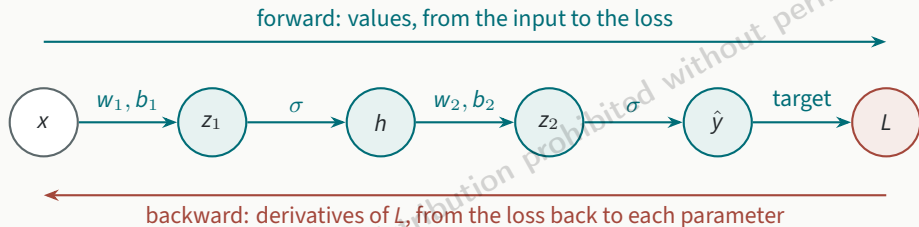
- ▶ The **loss** ℓ is a number that is small when the prediction $f_\theta(x_i)$ is close to the target y_i .
- ▶ θ has 10^3 to 10^9 entries, and $\mathcal{L}$ is not convex in θ . There is no formula for the minimiser.
- ▶ The only affordable information is the **gradient** $\nabla_{\theta} \mathcal{L}$: the direction in which the loss rises fastest.
- ▶ **Three questions organise this deck: how to get the gradient cheaply, what to do with it, and how to know the answer is worth anything.**

The Learning Loop: Forward, Loss, Backward, Update



- ▶ The **forward pass** evaluates the network, layer by layer, from input to prediction.
- ▶ The **backward pass** returns the derivative of the loss with respect to every parameter.
- ▶ The **learning rate** $\eta > 0$ is the step length: the one free choice in the update.
- ▶ **Every network in this course is trained by these four steps, repeated.**

One Graph, Two Traversals



- ▶ A **computation graph** records each intermediate value (node) and the operation behind it (edge).
- ▶ Backward walks the same edges in reverse, applying the **chain rule** edge by edge.
- ▶ The colour convention of this deck: **teal** and grey are values moving forward, **red** is derivatives moving back.
- ▶ **One graph, traversed twice. Backward reuses what forward stored.**

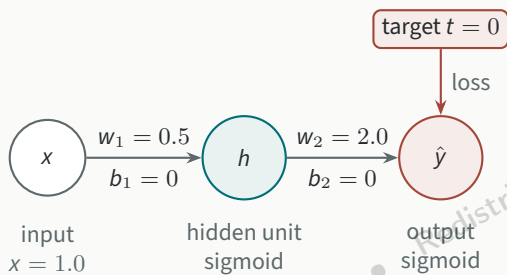
Session 2.3

Backpropagation, with Numbers

One observation, two layers, every derivative computed by hand

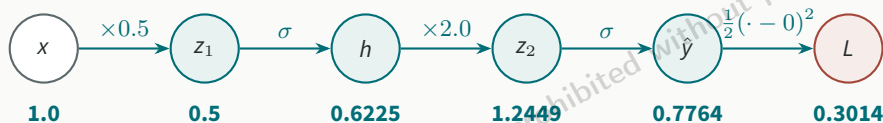


A Classifier, on One Observation



- ▶ The smallest honest version of the digit classifier: one input, one hidden unit, one output.
- ▶ $\hat{y} \in (0, 1)$ is the predicted probability that the image is a “3”. The target is $t = 0$: it is not.
- ▶ Loss: $L = \frac{1}{2}(\hat{y} - t)^2$, squared error, so that every step can be seen.
- ▶ **The arithmetic for one observation is the arithmetic for millions: forward, loss, backward, update.**

Forward, with Numbers

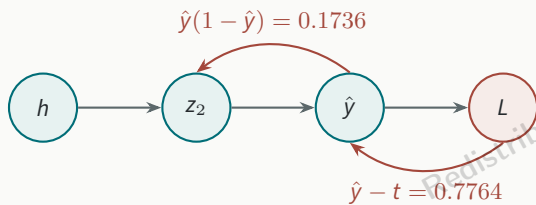


- ▶ $z_1 = w_1x + b_1 = 0.5$, and $h = \sigma(0.5) = 0.6225$: the hidden unit is “62% on”.
- ▶ $z_2 = w_2h + b_2 = 1.2449$, and $\hat{y} = \sigma(1.2449) = 0.7764$.
- ▶ $L = \frac{1}{2}(0.7764 - 0)^2 = 0.3014$. The network says “78% a 3”; the truth is that it is not.
- ▶ **Learning means moving w_1 and w_2 so that $\hat{y}$ falls. By how much each? That is the backward pass.**

Source: Session 2 lab notebook, block A2; every number is asserted there against PyTorch’s automatic differentiation.

Backward, Part One: the Output Layer

$$h = 0.6225$$

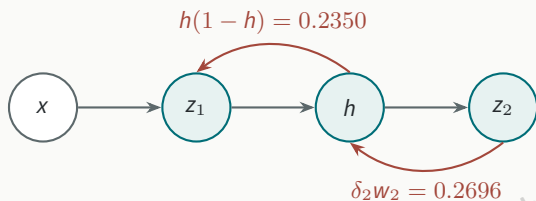


$$\delta_2 = \partial L / \partial z_2 = \mathbf{0.1348}$$

- ▶ Chain the links from w_2 to L :
$$\frac{\partial L}{\partial w_2} = \underbrace{(\hat{y} - t)}_{0.7764} \underbrace{\hat{y}(1 - \hat{y})}_{0.1736} \underbrace{h}_{0.6225}.$$
- ▶ Name the first two factors:
 $\delta_2 = \partial L / \partial z_2 = 0.1348$. It will be reused.
- ▶ Then $\partial L / \partial w_2 = \delta_2 h = 0.0839$ and $\partial L / \partial b_2 = \delta_2 = 0.1348$.
- ▶ **The sign is positive: raising w_2 raises the loss, so descent lowers w_2 .**

Source: Session 2 lab notebook, block A2.

Backward, Part Two: One Layer Further, and the Update

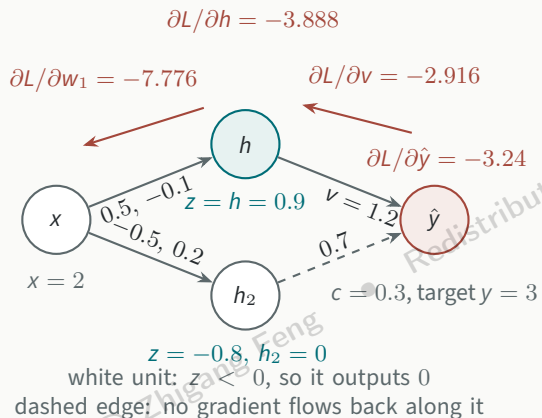


$$\delta_1 = \partial L / \partial z_1 = \mathbf{0.0633}$$

- ▶ Carry δ_2 back through w_2 :
 $\partial L / \partial h = \delta_2 w_2 = 0.2696$. Then through the activation:
 $\delta_1 = 0.2696 \times 0.2350 = 0.0633$.
- ▶ $\partial L / \partial w_1 = \delta_1 x = 0.0633$.
- ▶ Update with $\eta = 0.1$:
 $w_2 \leftarrow 2.0 - 0.1(0.0839) = 1.9916$ and
 $w_1 \leftarrow 0.5 - 0.1(0.0633) = 0.4937$.
- ▶ **Both sigmoid slopes are below 0.25. Each extra sigmoid layer shrinks the signal again: the vanishing gradient.**

Source: Session 2 lab notebook, block A2.

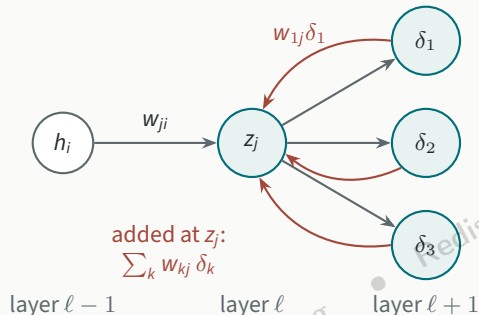
The Same Thing with ReLU — and a Unit That Gets No Gradient



- ▶ ReLU unit, $L = (\hat{y} - y)^2$. Forward: $\hat{y} = 1.2(0.9) + 0.3 = 1.38$, so $L = 2.6244$.
- ▶ Backward, in red: each derivative is the previous one times a local slope. ReLU's slope is 1: no shrinkage.
- ▶ $\eta = 0.01$: $w_1 \rightarrow 0.5778$, $v \rightarrow 1.2292$; the loss falls to 1.7488.
- ▶ **The second unit has $z < 0$: slope 0, gradient exactly 0. If that holds for every observation it never learns: a “dying ReLU”.**

Source: Miao & Feng, *AI for Economists*, §9.3.2; Session 2 lab notebook, block A2.

The Recursion, in General



- ▶ The **error signal** of a unit is $\delta_j = \partial L / \partial z_j$. At the output, $\delta = \hat{y} - y$ for squared error.
- ▶ **Back one layer:** add the messages from above, multiply by the local slope:
 $\delta_j = \sigma'(z_j) \sum_k w_{kj} \delta_k$.
- ▶ **A gradient:** $\partial L / \partial w_{ji} = \delta_j h_i$, signal at the weight's output end times activation at its input end.
- ▶ **Backpropagation is the chain rule with each intermediate derivative computed once and reused.**

Source: D. Rumelhart, G. Hinton, R. Williams, "Learning representations by back-propagating errors", *Nature* 323, 1986.

The Same Recursion, in Matrices

- ▶ Forward, for layers $\ell = 1, \dots, L$: $z^{(\ell)} = W_{\ell}h^{(\ell-1)} + b_{\ell}$ and $h^{(\ell)} = \sigma(z^{(\ell)})$, with $h^{(0)} = x$.
- ▶ Backward, from the top down ($\odot$ multiplies entry by entry):

$$\delta^{(\ell)} = (W_{\ell+1}^{\top} \delta^{(\ell+1)}) \odot \sigma'(z^{(\ell)}), \quad \frac{\partial L}{\partial W_{\ell}} = \delta^{(\ell)} (h^{(\ell-1)})^{\top}, \quad \frac{\partial L}{\partial b_{\ell}} = \delta^{(\ell)}.$$

- ▶ The backward pass uses the *transpose of the same matrix* that was used going forward.
- ▶ σ' multiplies at every layer, so the activation decides whether the signal survives depth.
- ▶ $h^{(\ell-1)}$ is needed on the way back, so forward activations must be stored: the memory cost of training.
- ▶ **Two lines of linear algebra per layer give every derivative in the network.**

Source: Miao & Feng, *AI for Economists*, §9.3, eqs. (9.20)–(9.21).

Backpropagation Is the Adjoint Method

- ▶ $\delta^{(\ell)}$ answers: by how much would the loss change if $z^{(\ell)}$ were perturbed? That is a **shadow price**.
- ▶ Treat each layer's equation $z^{(\ell)} = W_{\ell}h^{(\ell-1)} + b_{\ell}$ as a constraint. $\delta^{(\ell)}$ is its Lagrange multiplier.
- ▶ The backward recursion is a costate equation, solved from a terminal condition backwards, as in optimal control. There it is called the **adjoint method**; in numerical analysis, **reverse-mode differentiation**.
- ▶ A diagnosis follows: $\delta \approx 0$ in a layer means nothing below it can be improved at the current parameters. Printing gradient norms by layer is standard practice.
- ▶ **You already know how to read a multiplier. Backpropagation computes all of them in one backward sweep.**

Source: Miao & Feng, *AI for Economists*, §9.3.2; A. Griewank and A. Walther, *Evaluating Derivatives*, SIAM, 2008.

What Autograd Automates

- ▶ **Automatic differentiation** (autograd) builds the computation graph while the forward pass runs, then applies the recursion of the last three frames.
- ▶ The derivatives are exact to machine precision. Nothing is a finite difference, so there is no step size to choose and no truncation error to trade against rounding error.
- ▶ You write only the forward pass and the loss. No derivative is ever coded by hand.
- ▶ **Reverse mode** differentiates one scalar — the loss — with respect to all P parameters in a single backward sweep. That is exactly the shape of a training problem, and it is why the next frame's cost does not grow with P .
- ▶ **The forward pass is the only thing you write. Everything a gradient method needs follows from it.**

Source: Miao & Feng, *AI for Economists*, §9.3.4.

What a Gradient Costs

digits 784-128-10, $P = 101,770$, $n = 60,000$

 **1.90** × 187 → 355 ms

growth policy 1-32-32-1, $P = 1,153$, $n = 256$

 **3.33** × 0.11 → 0.36 ms

yield MLP 5-4-1, $P = 29$, $n = 300$

 **6.81** × 0.03 → 0.23 ms

Bar length: forward + backward per forward pass. Small networks: overhead, not arithmetic.

- ▶ Forward: about $2P$ operations per observation — a multiply and an add per weight. Backward returns two gradients per layer: for the weights, and for the layer's input.
- ▶ **All P derivatives at once cost a small multiple of one forward pass, whatever P is.** A numerical Jacobian needs $P + 1$ passes — $50,000 \times$ more at $P = 101,770$; a Hessian, P^2 numbers, impossible at $P = 10^6$.
- ▶ Textbooks say three passes. Measured: $1.59 \times$ with a leaf input, $2.90 \times$ when the input also needs a gradient; training never needs the first layer's, where nearly all the arithmetic is — about two.
- ▶ Budget $6PnE$: $P = 10^5$, $n = 5 \times 10^4$, $E = 100$ give 3×10^{12} operations, about 30 seconds on one core. Stored activations (batch $\times$ width $\times$ depth) cap the batch size, not statistics.
- ▶ **A macroeconomic network on a state vector is a small computation.**

Source: Measured on one core of an AMD EPYC 7763; Session 2 lab notebook, block A5. A. Griewank and A. Walther, *Evaluating Derivatives*, SIAM, 2008; J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., 2006; J. Kaplan et al., arXiv:2001.08361, 2020.

Session 2.3

The Loss, and the Optimizer

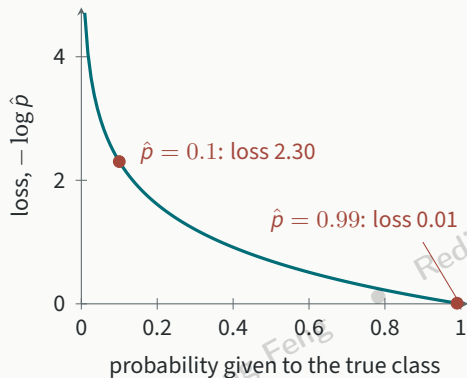
What the criterion assumes, and what to do with a gradient that is noisy



Loss I: Squared Error Is OLS

- ▶ For a continuous target, the mean squared error: $\mathcal{L} = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$.
- ▶ Minimising it is maximum likelihood under Gaussian errors. For a linear f_θ it is ordinary least squares.
- ▶ A loss is therefore an assumption about the noise, not a convention. Change the noise model and the loss changes: Poisson likelihood for counts, the check function for quantiles.
- ▶ In Deck 2.4 the target is next month's yield. In Deck 2.5 the "prediction" is an Euler-equation residual and the target is zero, so no data enter the loss at all.
- ▶ **Squared error asks for the conditional mean. If that is not what you want, it is the wrong loss.**

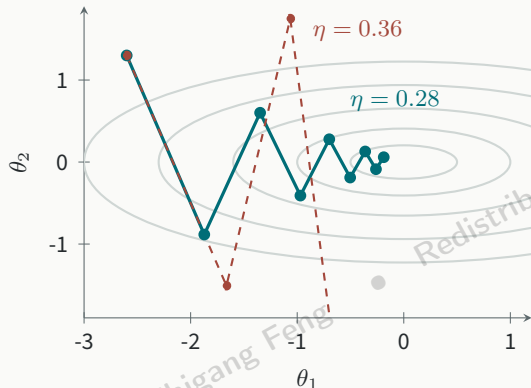
Loss II: Cross-Entropy Is McFadden



- ▶ A classifier returns a score z_k for each of K classes. The **softmax** turns scores into probabilities: $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$.
- ▶ **Cross-entropy** is the negative log-probability given to the true class, $\mathcal{L} = -\log \hat{p}_{k^*}$.
- ▶ Softmax is the logit choice probability, the label is the observed choice, and minimising cross-entropy maximises McFadden's multinomial-logit likelihood.
- ▶ **Confident and right costs nothing. Confident and wrong is unbounded.**

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

Gradient Descent: the Engine

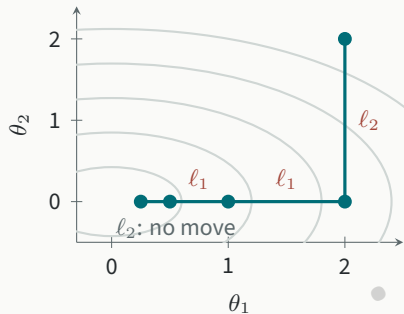


► Gradient descent:

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$. Step against the gradient.

- The picture is $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 6\theta_2^2)$. The teal path zigzags across the steep direction and converges.
- The red path uses a step only slightly longer. It overshoots further each time and diverges.
- **The learning rate is limited by the steepest direction, and progress by the flattest.**

Why a Sampled Gradient Still Gets There



- ▶ **Stochastic gradient descent** (SGD): differentiate a random subset of the loss's terms at each step.
- ▶ Here $\ell_1 = \frac{1}{2}\theta_1^2$, $\ell_2 = \theta_2^2$, step 0.5: only the drawn coordinate moves.
- ▶ Unbiased: its expectation is the full gradient.
- ▶ The surface is not convex, but with far more parameters than needed, most local minima are nearly as deep.
- ▶ **The path is not monotone and is not meant to be — and the goal is not the lowest training loss but a minimum that generalises.**

Source: L. Bottou, F. Curtis, J. Nocedal, *SIAM Rev.* 60(2), 2018; A. Choromanska et al., AISTATS 2015; T. Garipov et al., NeurIPS 2018.

Batch, Stochastic, Mini-Batch

Method	Terms per step	Cost of a step	Path
full batch	all n	high	smooth
stochastic	1	lowest	very noisy
mini-batch	32 to 1,024	one matrix product	noisy, averaged

- ▶ An **epoch** is one pass through the training data; an **iteration** is one update. With 40,000 observations and batches of 256, an epoch is 156 updates.
- ▶ Why mini-batch: many updates per pass, activations that fit in memory, hardware built for matrix products.
- ▶ The gradient noise also helps the iterate leave saddle points and shallow minima.
- ▶ **Count updates, not epochs: 100 epochs is 15,600 updates here and 600 on a sample of 1,600.**

Convergence: What Is Actually Guaranteed

- ▶ If the gradient is Lipschitz, its sampling noise has bounded variance, and the step sizes satisfy the **Robbins–Monro conditions**

$$\sum_n \eta_n = \infty \quad \text{and} \quad \sum_n \eta_n^2 < \infty,$$

then the expected squared gradient along the path goes to zero.

- ▶ $\sum \eta_n = \infty$, “go the distance”: the iterate can travel any distance from a bad start.
- ▶ $\sum \eta_n^2 < \infty$, “kill the noise”: accumulated sampling noise stays finite, so late steps stop bouncing.
- ▶ $\eta_n = 1/n$ satisfies both. A constant η fails the second: SGD then hovers in a ball around a solution.
- ▶ **The promise is a stationary point. Not a global minimum, not a rate, not a bound on the error.**

Source: H. Robbins and S. Monro, “A stochastic approximation method”, *Ann. Math. Statist.* 22(3), 1951; Bottou, Curtis, Nocedal (2018), §4.

Adaptive Step Sizes: RProp → RMSProp → Adam

- ▶ **Momentum**: step along an exponentially weighted average of past gradients. An economist would call it adaptive expectations about the descent direction.
- ▶ **RProp** (1993) sized each parameter's step from the sign of its gradient. **RMSProp** (2012) made that work with mini-batches: divide each step by a running average of that parameter's squared gradients.
- ▶ **Adam** (2015) is momentum plus RMSProp, with decay rates 0.9 and 0.999 by default.
- ▶ **AdamW** (2019) applies **weight decay**, a shrinkage of the weights at each step, outside the adaptive scaling. It is today's default. Later proposals: Lion, Sophia, schedule-free methods.
- ▶ **Adam is SGD with a step size for each parameter. It is still first-order: no Hessian is formed or stored.**

Source: D. Kingma and J. Ba, "Adam", ICLR 2015; I. Loshchilov and F. Hutter, "Decoupled weight decay regularization", ICLR 2019; M. Riedmiller and H. Braun, IEEE ICNN 1993.

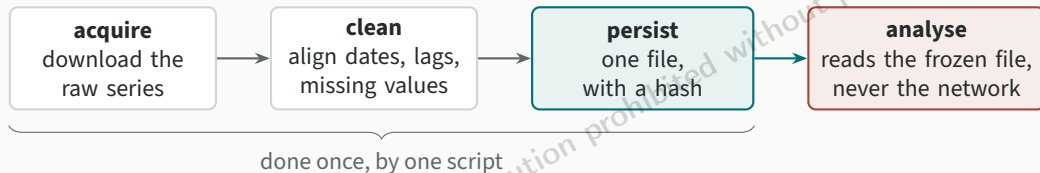
Session 2.3

The Dataset, and Three Samples

Where the data come from, and the rule that makes a reported number mean something



The Data Workflow: Acquire, Clean, Persist — Once



- ▶ Statistical agencies revise. A notebook that downloads on every run gives different answers on different days.
- ▶ Separate the download from the analysis; the analysis then runs offline and gives the same numbers every time.
- ▶ **A result is reproducible only if its data are frozen.**

Train, Validation, Test: Three Jobs, Three Samples

training sample: choose the parameters θ

the optimiser sees only this

validation

choose the design:
width, penalty,
when to stop

test

report once,
at the end

- ▶ A flexible model can fit its training sample as closely as you like. That says nothing about new data.
- ▶ **Validation** data choose between designs, so they are no longer a clean test.
- ▶ **Test** data are touched once. Their error estimates the error on future data.
- ▶ With time series the three samples are consecutive periods, never a shuffle.
- ▶ **Each sample does one job. A sample used for two jobs does neither honestly.**

Leakage: Four Ways the Test Set Gets into Training

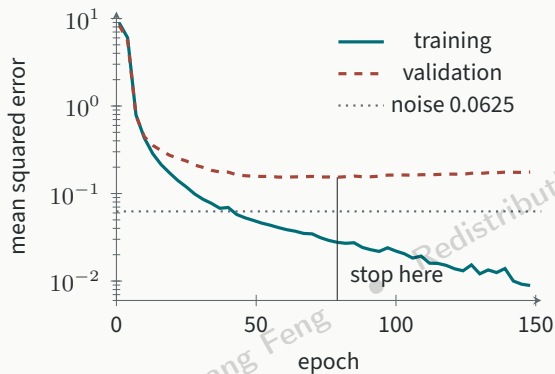
- ▶ **Leakage:** information from the test sample reaches the fitted model, so the test error is too good.
- ▶ **Shuffling a time series.** Neighbouring months are nearly identical; a shuffled split puts the answer for March next to the question for April.
- ▶ **Scaling with full-sample statistics.** The mean and standard deviation used to standardise the inputs must come from the training window alone.
- ▶ **Tuning on the test set.** Trying ten designs and reporting the best test error makes the test set a validation set.
- ▶ **The same template on both sides.** In the lab, sentences built as subject $\times$ phrase score 0.97 on a random split and 0.50 when whole phrases are held out.
- ▶ **Ask of every reported accuracy: what did the model already know about these cases?**

Source: Accuracies: Session 2 lab notebook, appendix (logistic regression on word counts, 96 sentences).

Validation Is the Part You Cannot Delegate

- ▶ An AI assistant will write every line of Deck 2.4's code, correctly, in seconds.
- ▶ It cannot tell you whether the result is *right*, because it does not know what right means for your problem: which benchmark, which tolerance, which sample is held out.
- ▶ **You supply:** the question, the model, the information set, the accuracy criterion.
- ▶ **The tool supplies:** syntax, the translation into code, the routines, the plots.
- ▶ A training loop that runs without error has told you nothing yet. A training loop that reproduces a known answer has told you the loop is right.
- ▶ **The bottleneck moved from writing the loop to saying what the loop must satisfy.**

Overfitting, Underfitting, and When to Stop



- **Underfitting:** both curves high.
- **Overfitting:** training loss falls while validation loss turns up.
- $n = 2,000$. Training loss ends at 0.0085, below the noise variance 0.0625: memorised noise.
- **Early stopping:** keep the best-validation parameters, near epoch 80.
- **A training loss below the noise floor is a diagnosis, not an achievement.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

Three Instruments Against Overfitting, and What They Are Worth

Strategy	best validation	
	$n = 2,000$	$n = 50,000$
none	0.1535	0.0788
weight decay 10^{-4}	0.1536	0.0804
dropout 0.1	0.1257	0.0780
dropout 0.2	0.1341	0.0792
both (0.1)	0.1238	0.0780

Noise floor 0.0625; 30 inputs, 17 pure noise.

Source: Recorded in the instructor's quantitative-macro course computations (2026), median of three seeds; not recomputed in this session's lab.

- ▶ **Weight decay:** a penalty $\lambda \|\theta\|^2$, which is ridge regression.
- ▶ **Dropout:** set a random share of units to zero at each training step. It averages many sub-networks.
- ▶ Early stopping acts like a ridge penalty of about $1/(\eta T)$ after T steps.
- ▶ **Dropout pays where data are scarce: 18% at $n = 2,000$, 1% at $n = 50,000$. More data does more: 0.1535 to 0.0788.**

Takeaways, and What Is Still Missing

- ▶ **Backpropagation** is the chain rule with intermediate derivatives named and reused. Those derivatives are shadow prices.
- ▶ **The optimiser is first-order because it must be:** a Hessian at $P = 10^6$ has 10^{12} entries. Adam is SGD with a step size for each parameter.
- ▶ **What is guaranteed** is a stationary point under the Robbins–Monro conditions, and nothing stronger.
- ▶ **Each sample does one job:** the optimiser sees the training data, the design is chosen on validation data, and the test data are touched once.
- ▶ **Generalisation is measured, not assumed:** a training loss below the noise floor is the mark of overfitting, and regularisation pays mainly when data are scarce. Deck 2.6 returns with a case the noise floor cannot diagnose at all.
- ▶ **Next, Deck 2.4: the same loop in PyTorch, first where the answer is known, then on next month's 10-year yield.**